Sprint 24 Structured Notes: Graphs, Traversal, Trees, Tries, Priority Queues, and Heaps

0. What Sprint 24 Is About

Sprint 24 is about **non-linear data structures**.

Before this sprint, you learned many structures where data mostly moves in a line:

- arrays
- stacks
- queues
- linked lists

Those are useful, but many real-world problems are not shaped like a straight line.

For example:

- people are connected to other people
- cities are connected by roads
- folders contain folders
- websites link to other websites
- words share prefixes
- urgent tasks should happen before less important tasks

Sprint 24 teaches structures that can represent those situations:

- graphs
- graph traversal
- breadth-first search, or BFS
- depth-first search, or DFS
- trees
- binary trees
- binary search trees
- tries
- priority queues
- heaps

The main goal is:

> Model relationships, hierarchies, and priority-based data.

Simple meaning:

> Sprint 24 teaches you how to store data when the data branches, connects, or has importance levels.

---

1. Current Progress Before Sprint 24

Before Sprint 24, you should already understand several important ideas.

Earlier topic	What you learned	How it helps in Sprint 24
Variables	Store values with `let` and `const`	Graphs, trees, and heaps are stored in variables.
Objects	Store key-value data	Graphs are often represented with objects or maps.
Arrays	Store ordered lists	Neighbor lists, queues, stacks, and heaps often use arrays.
Sets	Store unique values	Graph traversal uses `Set` to remember visited nodes.
Maps	Store flexible key-value pairs	Graphs can be represented with `Map`.
Functions	Reuse logic	Traversal algorithms are usually functions.
Recursion	A function calls itself	DFS and tree traversal often use recursion.
Stacks	Last In, First Out	DFS can use a stack.
Queues	First In, First Out	BFS uses a queue.
Big O	Compare efficiency	Sprint 24 compares traversal and heap operation costs.
Classes	Create object blueprints	Trees, priority queues, and heaps can be written as classes.

Simple summary:

> Sprint 23 taught you basic data containers. Sprint 24 teaches you connected and hierarchical containers.

---

2. Main Mental Shift in Sprint 24

Earlier data structures were mostly linear.

Example array:

```
const numbers = [10, 20, 30, 40];
```

This is linear:

```
10 -> 20 -> 30 -> 40
```

But a graph can branch:

```
A -- B
|    |
C -- D
```

A tree can also branch:

```
Root
├── Child 1
└── Child 2
```

A heap is also tree-like:

```
      1
     /
    3   5
```

The main question changes.

Before Sprint 24:

> How do I store a list?

In Sprint 24:

> How do I store relationships, hierarchy, and priority?

Part A — Graphs

3. What Is a Graph?

A **graph** is a data structure made of:

Word	Meaning
Node	A thing in the graph
Vertex	Another word for node
Edge	A connection between nodes

Example graph:

```
A -- B
|    |
C -- D
```

In this graph:

```
A, B, C, D
```

are nodes.

The lines between them are edges.

Simple meaning:

> A graph stores things and the relationships between those things.

Real-world examples:

Real-world situation	Graph meaning
Friends on social media	People are nodes. Friendships are edges.
Roads between cities	Cities are nodes. Roads are edges.
Website links	Pages are nodes. Links are edges.
Computer networks	Devices are nodes. Connections are edges.
Course prerequisites	Courses are nodes. Prerequisite relationships are edges.

---

4. Graph Vocabulary

4.1 Node / Vertex

A node is one item in the graph.

Example:

```
A
```

Simple meaning:

> A node is one thing.

4.2 Edge

An edge is a connection between two nodes.

Example:

```
A -- B
```

This means A is connected to B.

Simple meaning:

> An edge is a relationship.

4.3 Neighbor

A neighbor is a node directly connected to another node.

Example:

```
A -- B
A -- C
```

B and C are neighbors of A.

Simple meaning:

> A neighbor is directly reachable in one step.

4.4 Path

A path is a sequence of connected nodes.

Example:

```
A -> B -> D
```

Simple meaning:

> A path is a route from one node to another.

4.5 Cycle

A cycle happens when you can leave a node and eventually come back to it.

Example:

```
A -- B
|    |
C -- D
```

One cycle is:

```
A -> B -> D -> C -> A
```

Simple meaning:

> A cycle is a loop inside a graph.

---

5. Common Graph Types

5.1 Undirected Graph

An **undirected graph** has connections that go both ways.

Example:

```
A -- B
```

If A is connected to B, then B is also connected to A.

Real-world example:

> Friendship. If Ada is friends with Linus, Linus is friends with Ada.

JavaScript representation:

```
const graph = {
  A: ["B"],
  B: ["A"]
};
```

Simple meaning:

> Undirected means both directions are allowed.

5.2 Directed Graph

A **directed graph** has connections with direction.

Example:

```
A -> B
```

This means A points to B.

It does not automatically mean B points back to A.

Real-world example:

> Following someone on social media. You can follow someone without them following you back.

JavaScript representation:

```
const follows = {
  Sangeeth: ["Ada"],
  Ada: ["Linus"],
  Linus: []
};
```

This means:

```
Sangeeth -> Ada -> Linus
```

Simple meaning:

> Directed means the connection has an arrow.

5.3 Weighted Graph

A **weighted graph** has edges with values.

Example:

```
A --5-- B
```

The edge between A and B has weight 5.

The weight can mean:

- distance
- cost
- time
- difficulty
- priority

Real-world example:

> Roads between cities. The weight can be distance in kilometers.

JavaScript representation:

```
const graph = {
  A: [{ node: "B", weight: 5 }],
```

```
  B: [{ node: "A", weight: 5 }]
};
```

Simple meaning:

> Weighted means each connection has a number or cost attached to it.

5.4 Cyclic Graph

A **cyclic graph** contains at least one cycle.

Example:

```
A -- B
|    |
C -- D
```

You can go:

```
A -> B -> D -> C -> A
```

Simple meaning:

> Cyclic means the graph has a loop.

5.5 Acyclic Graph

An **acyclic graph** has no cycles.

Example:

```
A -> B -> C
```

There is no way to follow edges and return to the same node.

Simple meaning:

> Acyclic means no loops.

5.6 DAG: Directed Acyclic Graph

A **DAG** means:

```
Directed Acyclic Graph
```

That means:

- edges have direction
- there are no cycles

Example:

```
A -> B -> C
A -> D
```

Real-world examples:

- task dependencies
- course prerequisites
- build systems
- version histories

Simple meaning:

> A DAG is a directed graph that does not loop back.

5.7 Disconnected Graph

A **disconnected graph** has some nodes that cannot reach each other.

Example:

```
A -- B     C -- D
```

A and B are connected to each other.

C and D are connected to each other.

But A cannot reach C.

Simple meaning:

> Disconnected means the graph has separate parts.

Part B — Graph Representation

6. Why Graph Representation Matters

A graph is an idea.

To use it in JavaScript, you need a way to store it.

Two common ways are:

- adjacency list
- adjacency matrix

Simple meaning:

> Representation means how the graph is stored in code.

---

7. Adjacency List

An **adjacency list** stores each node with a list of its neighbors.

Example graph:

```
A -- B
|
C
```

JavaScript representation:

```
const graph = {
  A: ["B", "C"],
  B: ["A"],
  C: ["A"]
};
```

Read this:

```
A: ["B", "C"]
```

as:

```
A is connected to B and C.
```

Simple meaning:

> An adjacency list stores neighbors.

---

8. Adjacency List with Map

You can also use `Map`.

```
const graph = new Map();

 graph.set("A", ["B", "C"]);
 graph.set("B", ["A"]);
 graph.set("C", ["A"]);
```

Using `Map` can be useful when keys are not simple strings.

Simple beginner rule:

> Use objects for simple graph practice. Use `Map` when you need more flexibility.

---

9. Adjacency Matrix

An **adjacency matrix** stores connections in a grid.

Example:

```
    A B C
A [ 0 1 1 ]
B [ 1 0 0 ]
C [ 1 0 0 ]
```

This means:

- A connects to B and C
- B connects to A
- C connects to A

A `1` means connected.

A `0` means not connected.

JavaScript representation:

```
const matrix = [
  [0, 1, 1],
  [1, 0, 0],
  [1, 0, 0]
];

const nodes = ["A", "B", "C"];
```

Simple meaning:

> An adjacency matrix is a table of connections.

---

10. Adjacency List vs Adjacency Matrix

Representation	Strength	Weakness
Adjacency list	Saves space when there are not many edges	Checking one exact edge can be slower
Adjacency matrix	Fast to check if two nodes connect	Can waste space when most nodes are not connected

Beginner rule:

> Use adjacency lists first. They are easier and usually better for beginner graph problems.

---

11. Sparse Graph and Dense Graph

Sparse Graph

A sparse graph has relatively few edges.

Example:

```
A -- B     C     D -- E
```

Most nodes are not connected to many other nodes.

Adjacency lists are usually good for sparse graphs.

Dense Graph

A dense graph has many edges.

Example:

```
A connected to B, C, D, E
B connected to A, C, D, E
C connected to A, B, D, E
```

Many nodes connect to many other nodes.

Adjacency matrices may be more reasonable for dense graphs.

Simple rule:

> Few connections: adjacency list. Many connections: adjacency matrix may be useful.

Part C — Graph Traversal

12. What Is Traversal?

Traversal means visiting nodes in a graph or tree.

Simple meaning:

> Traversal is moving through a data structure step by step.

Example graph:

```
A -- B
|
C
```

Starting from A, you may visit:

```
A, B, C
```

Graph traversal is used for:

- finding whether a node exists
- finding connected nodes
- finding paths
- checking relationships
- searching networks
- exploring trees

---

13. The Two Main Traversal Methods

Sprint 24 focuses on two important traversal methods:

Method	Full name	Simple meaning	Main tool
BFS	Breadth-First Search	Visit nearby nodes first	Queue
DFS	Depth-First Search	Go deep first	Stack or recursion

Simple comparison:

> BFS spreads out. DFS digs down.

Part D — BFS: Breadth-First Search

14. What Is BFS?

BFS means **Breadth-First Search**.

Simple meaning:

> BFS visits the closest nodes first, then moves farther away.

Example graph:

```
   A
  /
B   C
|
D
```

Starting at A, BFS visits:

```
A, B, C, D
```

Why?

1. Visit A.
2. Visit A's neighbors: B and C.
3. Then visit the next level: D.

BFS is called breadth-first because it explores across the width before going deeper.

---

15. BFS Uses a Queue

BFS uses a queue.

A queue follows FIFO:

```
First In, First Out
```

This means the first node added is the first node removed.

Queue example:

```
Add A
Add B
Add C
Remove A first
Then B
Then C
```

Simple meaning:

BFS waits in line. Nodes are processed in the order they were discovered.

---

16. BFS Code

```
const graph = {
  A: ["B", "C"],
  B: ["A", "D"],
  C: ["A"],
  D: ["B"]
};

function bfs(graph, start) {
  const visited = new Set();
  const queue = [start];
  const order = [];

  visited.add(start);

  while (queue.length > 0) {
    const node = queue.shift();
    order.push(node);

    for (const neighbor of graph[node] ?? []) {
      if (!visited.has(neighbor)) {
        visited.add(neighbor);
        queue.push(neighbor);
      }
    }
  }

  return order;
}

console.log(bfs(graph, "A"));
```

Expected output:

```
["A", "B", "C", "D"]
```

---

17. BFS Code Explained

```
const visited = new Set();
```

This stores nodes that have already been seen.

It prevents repeated visits.

```
const queue = [start];
```

This creates a queue and puts the starting node inside it.

```
const order = [];
```

This stores the visit order.

```
visited.add(start);
```

This marks the starting node as visited.

```
while (queue.length > 0) {
```

This keeps running while there are nodes waiting in the queue.

```
const node = queue.shift();
```

This removes the first node from the queue.

`shift()` removes from the front of the array.

```
for (const neighbor of graph[node] ?? []) {
```

This loops through the current node's neighbors.

`?? []` means:

> If `graph[node]` is missing or undefined, use an empty array instead.

This prevents errors.

```
if (!visited.has(neighbor)) {
```

This checks whether the neighbor has not been visited yet.

```
visited.add(neighbor);
queue.push(neighbor);
```

This marks the neighbor as visited and adds it to the queue.

---

18. BFS Dry Run

Graph:

```
const graph = {
  A: ["B", "C"],
  B: ["A", "D"],
  C: ["A"],
  D: ["B"]
};
```

Start:

```
queue = [A]
visited = {A}
order = []
```

Step 1:

```
Remove A
order = [A]
Add B and C
queue = [B, C]
visited = {A, B, C}
```

Step 2:

```
Remove B
order = [A, B]
A is already visited
Add D
queue = [C, D]
visited = {A, B, C, D}
```

Step 3:

```
Remove C
order = [A, B, C]
A is already visited
queue = [D]
```

Step 4:

```
Remove D
order = [A, B, C, D]
B is already visited
queue = []
```

Final result:

```
["A", "B", "C", "D"]
```

---

19. Why `visited` Matters

Graphs can contain cycles.

Example:

```
A -- B
```

A connects to B.

B connects back to A.

Without `visited`, traversal could repeat forever:

```
A -> B -> A -> B -> A -> B ...
```

This is why graph traversal usually needs:

```
const visited = new Set();
```

Beginner rule:

> In graph traversal, use a `visited` Set unless you are completely sure there are no cycles.

---

20. When BFS Is Useful

BFS is useful when:

- you need level-by-level traversal
- you need the shortest path in an unweighted graph
- you want to find nearby nodes first
- you are searching social connections by degree
- you are exploring a tree level by level

Example:

> Find the fewest number of connections between two people in an unweighted social graph.

BFS is good because it checks closer connections before farther connections.

Part E — DFS: Depth-First Search

21. What Is DFS?

DFS means **Depth-First Search**.

Simple meaning:

> DFS goes as deep as possible before coming back.

Example graph:

```
   A
  /
B   C
|
D
```

Starting at A, DFS might visit:

```
A, B, D, C
```

Why?

1. Start at A.
2. Go to B.
3. From B, go to D.
4. D has no new neighbors.
5. Go back and visit C.

DFS is called depth-first because it follows one path deeply before trying another path.

---

22. DFS Can Use Recursion or a Stack

DFS can be implemented in two common ways:

Method	Meaning
Recursion	The function calls itself for each neighbor.
Stack	Store nodes manually using Last In, First Out behavior.

Beginner rule:

> Recursive DFS is easier to read first. Stack DFS helps you understand what recursion is doing internally.

---

23. DFS Code with Recursion

```
const graph = {
  A: ["B", "C"],
  B: ["A", "D"],
  C: ["A"],
  D: ["B"]
};

function dfs(graph, start, visited = new Set(), order = []) {
  if (visited.has(start)) {
    return order;
  }

  visited.add(start);
  order.push(start);

  for (const neighbor of graph[start] ?? []) {
    dfs(graph, neighbor, visited, order);
  }

  return order;
}

console.log(dfs(graph, "A"));
```

Possible output:

```
["A", "B", "D", "C"]
```

The exact output can change depending on the neighbor order.

---

24. DFS Code Explained

```
function dfs(graph, start, visited = new Set(), order = []) {
```

This function receives:

Parameter	Meaning
`graph`	The graph to search

Parameter	Meaning
`start`	The current node
`visited`	Nodes already visited
`order`	Visit order

```
if (visited.has(start)) {
  return order;
}
```

If the node was already visited, stop this path.

```
visited.add(start);
order.push(start);
```

Mark the node as visited and record it.

```
for (const neighbor of graph[start] ?? []) {
  dfs(graph, neighbor, visited, order);
}
```

For each neighbor, recursively visit that neighbor.

```
return order;
```

Return the final traversal order.

---

25. DFS Dry Run

Graph:

```
const graph = {
  A: ["B", "C"],
  B: ["A", "D"],
```

```
  C: ["A"],
  D: ["B"]
};
```

Start at A.

Step 1:

```
Visit A
order = [A]
```

A's neighbors are B and C.

DFS goes to B first.

Step 2:

```
Visit B
order = [A, B]
```

B's neighbors are A and D.

A is already visited.

DFS goes to D.

Step 3:

```
Visit D
order = [A, B, D]
```

D's neighbor is B.

B is already visited.

Return back.

Step 4:

```
Go back to A
Visit C
order = [A, B, D, C]
```

Final result:

```
["A", "B", "D", "C"]
```

---

26. DFS with an Explicit Stack

DFS can also use a stack.

```
function dfsWithStack(graph, start) {
  const visited = new Set();
  const stack = [start];
  const order = [];

  while (stack.length > 0) {
    const node = stack.pop();

    if (visited.has(node)) {
      continue;
    }

    visited.add(node);
    order.push(node);

    for (const neighbor of graph[node] ?? []) {
      if (!visited.has(neighbor)) {
        stack.push(neighbor);
      }
    }
  }

  return order;
}

console.log(dfsWithStack(graph, "A"));
```

Simple meaning:

> A stack makes DFS go deep because the most recently added node is processed first.

27. BFS vs DFS

Question	Better choice
Need shortest path in an unweighted graph?	BFS
Need level-by-level order?	BFS
Need to explore one path deeply?	DFS
Need simple recursive tree traversal?	DFS
Need to check all nodes?	Either BFS or DFS

Simple comparison:

BFS	DFS
Uses a queue	Uses recursion or stack
Visits nearby nodes first	Goes deep first
Good for shortest unweighted path	Good for deep exploration
Spreads out	Digs down

Beginner memory trick:

```
BFS = Broad / level by level
DFS = Deep / path by path
```

28. Graph Traversal Complexity

For an adjacency list:

```
Time: O(V + E)
Space: O(V)
```

Where:

Symbol	Meaning
`V`	Number of vertices / nodes

Symbol	Meaning
`E`	Number of edges

Why time is `O(V + E)`:

- each node is visited once
- each edge is checked during traversal

Why space is `O(V)`:

- `visited` can store all nodes
- queue or stack can also store nodes

Simple meaning:

> BFS and DFS usually visit all nodes and edges that are reachable from the start.

Part F — Trees

29. What Is a Tree?

A **tree** is a special type of graph.

Simple meaning:

> A tree is a hierarchy.

Example:

```
CEO
├── Engineering Manager
│   ├── Developer
│   └── QA Engineer
└── Sales Manager
    └── Sales Rep
```

Real-world tree examples:

- family trees
- folder systems
- HTML DOM tree
- organization charts

- category menus
- decision trees

---

30. Tree Vocabulary

Word	Meaning
Root	The top node of the tree
Parent	A node above another node
Child	A node below another node
Sibling	Nodes with the same parent
Leaf	A node with no children
Subtree	A smaller tree inside a tree
Edge	Connection between parent and child
Depth	How far a node is from the root
Height	Longest path from a node down to a leaf

---

31. Root

The root is the top node.

Example:

```
Root
├── Child A
└── Child B
```

Simple meaning:

> The root is where the tree starts.

---

32. Parent and Child

Example:

```
Parent
├── Child 1
└── Child 2
```

The parent is above.

The children are below.

Simple meaning:

> Parent-child relationships create the tree hierarchy.

---

33. Leaf

A leaf is a node with no children.

Example:

```
Root
└── Leaf
```

Simple meaning:

> A leaf is an ending node.

---

34. Subtree

A subtree is part of a tree that is itself a tree.

Example:

```
A
├── B
│   ├── D
│   └── E
└── C
```

The section starting at B is a subtree:

```
B
├── D
└── E
```

Simple meaning:

> A subtree is a smaller tree inside a bigger tree.

---

35. Tree Representation in JavaScript

A tree can be represented with nested objects.

```
const tree = {
  name: "Grandparent",
  children: [
    {
      name: "Parent 1",
      children: [
        { name: "Child 1", children: [] },
        { name: "Child 2", children: [] }
      ]
    },
    {
      name: "Parent 2",
      children: []
    }
  ]
};
```

Each node has:

```
name
children
```

Simple meaning:

> A tree node can store data and a list of child nodes.

---

36. Tree Traversal with Recursion

Trees are often traversed with recursion.

```
function printTree(node) {
  console.log(node.name);

  for (const child of node.children) {
    printTree(child);
  }
}

printTree(tree);
```

Simple meaning:

1. Print the current node.
2. For each child, print that child's tree.

This works because every child can be treated like the root of a smaller tree.

---

37. Preorder Traversal

Preorder means:

```
Visit node first, then children.
```

Pattern:

```
Node -> Left/Children
```

Example:

```
function preorder(node) {
  if (node === null) {
    return;
  }

  console.log(node.value);
  preorder(node.left);
```

```
  preorder(node.right);
}
```

Simple meaning:

> Preorder processes the parent before the children.

---

38. Inorder Traversal

Inorder is mostly used with binary trees.

Pattern:

```
Left -> Node -> Right
```

Example:

```
function inorder(node) {
  if (node === null) {
    return;
  }

  inorder(node.left);
  console.log(node.value);
  inorder(node.right);
}
```

Simple meaning:

> Inorder processes the left side, then the node, then the right side.

For a binary search tree, inorder traversal gives values in sorted order.

---

39. Postorder Traversal

Postorder means:

```
Children first, then node.
```

Pattern:

```
Left -> Right -> Node
```

Example:

```
function postorder(node) {
  if (node === null) {
    return;
  }

  postorder(node.left);
  postorder(node.right);
  console.log(node.value);
}
```

Simple meaning:

> Postorder processes children before the parent.

This can be useful when deleting or cleaning up a tree.

Part G — Binary Trees and Binary Search Trees

40. What Is a Binary Tree?

A **binary tree** is a tree where each node has at most two children.

The two children are usually called:

- left child
- right child

Example:

```
      10
     /
    5     15
```

Simple meaning:

> A binary tree allows up to two children per node.

---

41. Binary Tree Node Representation

A binary tree node can look like this:

```
const node = {
  value: 10,
  left: null,
  right: null
};
```

A larger tree:

```
const tree = {
  value: 10,
  left: {
    value: 5,
    left: null,
    right: null
  },
  right: {
    value: 15,
    left: null,
    right: null
  }
};
```

Simple meaning:

> Each node stores a value and links to left and right children.

---

42. What Is a Binary Search Tree?

A **binary search tree**, or **BST**, is a binary tree with an ordering rule.

Rule:

```
Left side = smaller values
Right side = larger values
```

Example:

```
        10
       /
      5     15
     / \
    2   7      20
```

For every node:

- values on the left are smaller
- values on the right are larger

Simple meaning:

> A BST is a binary tree organized for faster searching.

---

43. Why BSTs Are Useful

A BST can skip large parts of the tree.

Search for 7:

```
Start at 10.
7 is smaller than 10, so go left.
Now at 5.
7 is larger than 5, so go right.
Found 7.
```

You did not check every node.

Simple meaning:

> A BST uses order to guide the search.

---

44. BST Search Code

```
const bst = {
  value: 10,
  left: {
    value: 5,
    left: { value: 2, left: null, right: null },
    right: { value: 7, left: null, right: null }
```

```
  },
  right: {
    value: 15,
    left: null,
    right: { value: 20, left: null, right: null }
  }
};

function searchBST(node, target) {
  if (node === null) {
    return false;
  }

  if (node.value === target) {
    return true;
  }

  if (target < node.value) {
    return searchBST(node.left, target);
  }

  return searchBST(node.right, target);
}

console.log(searchBST(bst, 7));
console.log(searchBST(bst, 99));
```

Expected output:

```
true
false
```

---

45. BST Search Explained

```
if (node === null) {
  return false;
}
```

If we reach an empty place, the target is not in the tree.

```
if (node.value === target) {
  return true;
}
```

If the current node is the target, return true.

```
if (target < node.value) {
  return searchBST(node.left, target);
}
```

If the target is smaller, search the left side.

```
return searchBST(node.right, target);
```

If the target is larger, search the right side.

---

46. Balanced vs Unbalanced BST

A balanced BST is reasonably even.

Example:

```
      10
     /
    5     15
   / \    /
  2   7  12  20
```

Searching is efficient because each step removes about half the remaining tree.

An unbalanced BST can look like a linked list.

Example:

```
1
```

```
  2

    3

      4
```

This is weaker because you may have to move one node at a time.

Simple warning:

> A BST is efficient only when it is reasonably balanced.

---

47. BST Complexity

Operation	Balanced BST	Unbalanced BST
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)

Simple meaning:

> A good BST can be fast. A badly shaped BST can act like a linked list.

Part H — Tries

48. What Is a Trie?

A **trie** is a tree used to store strings by characters.

Simple meaning:

> A trie stores words one letter at a time.

Example words:

```
cat
car
care
```

Trie idea:

```
c
└── a
    ├── t
    └── r
        └── e
```

The words share the prefix:

```
ca
```

So the trie stores that shared prefix once.

---

49. Why Tries Are Useful

Tries are useful for:

- autocomplete
- prefix search
- spell checking
- dictionary lookup
- word games
- search suggestions

Example:

```
Words: cat, car, care
Prefix: ca
Matches: cat, car, care
```

Simple meaning:

> A trie is useful when many strings share starting letters.

---

50. Trie Node

A trie node usually stores:

Property	Meaning
`children`	Next possible letters
`isWord`	Whether this node completes a full word

Example:

```
function createTrieNode() {
  return {
    children: {},
    isWord: false
  };
}
```

Simple meaning:

> A trie node knows its next letters and whether it ends a word.

---

51. Insert into a Trie

```
function createTrieNode() {
  return {
    children: {},
    isWord: false
  };
}

const root = createTrieNode();

function insert(word) {
  let current = root;

  for (const char of word) {
    if (!current.children[char]) {
      current.children[char] = createTrieNode();
    }

    current = current.children[char];
  }

  current.isWord = true;
}
```

```
insert("cat");
insert("car");
```

Simple meaning:

1. Start at the root.
2. Look at each character.
3. Create a node if the character path does not exist.
4. Move to that character node.
5. Mark the last node as a complete word.

---

52. Search a Trie

```
function search(word) {
  let current = root;

  for (const char of word) {
    if (!current.children[char]) {
      return false;
    }

    current = current.children[char];
  }

  return current.isWord;
}

console.log(search("cat"));
console.log(search("ca"));
console.log(search("dog"));
```

Expected output:

```
true
false
false
```

Why does `"ca"` return false?

Because `"ca"` is only a prefix.

It was not marked as a full word.

This line marks a full word:

```
current.isWord = true;
```

---

53. Prefix Search in a Trie

Sometimes you do not need to know if something is a full word.

You only need to know if a prefix exists.

```
function startsWith(prefix) {
  let current = root;

  for (const char of prefix) {
    if (!current.children[char]) {
      return false;
    }

    current = current.children[char];
  }

  return true;
}

console.log(startsWith("ca"));
console.log(startsWith("do"));
```

Possible output:

```
true
false
```

Simple meaning:

`search()` checks complete words. `startsWith()` checks prefixes.

---

54. Trie Complexity

Let `L` be the length of the word.

Operation	Complexity
Insert word	O(L)
Search word	O(L)
Prefix check	O(L)

Simple meaning:

> Trie operations depend on the length of the word, not directly on the number of stored words.

Part I — Priority Queues

55. What Is a Priority Queue?

A normal queue removes the oldest item first.

Normal queue rule:

```
First In, First Out
```

A priority queue removes the most important item first.

Simple meaning:

> A priority queue serves items by importance, not only by arrival order.

Example:

```
Normal queue:
Task A, Task B, Task C

Priority queue:
Urgent task first, even if it arrived later.
```

---

56. Priority Queue Examples

Priority queues are used in:

- emergency rooms
- task schedulers
- printer queues
- operating systems
- game AI
- pathfinding algorithms
- event simulation

Example task data:

```
const tasks = [
  { name: "Clean room", priority: 3 },
  { name: "Submit assignment", priority: 1 },
  { name: "Play game", priority: 5 }
];
```

Assume smaller number means higher priority:

```
1 = highest priority
5 = lowest priority
```

So this task should be processed first:

```
Submit assignment
```

---

57. Simple Priority Queue with Array Sorting

```
class PriorityQueue {
  constructor() {
    this.items = [];
    this.counter = 0;
  }

  enqueue(element, priority) {
    this.items.push({
      element,
      priority,
```

```
      order: this.counter
    });

    this.counter++;

    this.items.sort((a, b) => {
      return a.priority - b.priority || a.order - b.order;
    });
  }

  dequeue() {
    if (this.items.length === 0) {
      return null;
    }

    return this.items.shift().element;
  }

  peek() {
    if (this.items.length === 0) {
      return null;
    }

    return this.items[0].element;
  }
}

const pq = new PriorityQueue();

pq.enqueue("Clean room", 3);
pq.enqueue("Submit assignment", 1);
pq.enqueue("Play game", 5);

console.log(pq.dequeue());
console.log(pq.dequeue());
console.log(pq.dequeue());
```

Expected output:

```
Submit assignment
Clean room
Play game
```

---

58. Priority Queue Code Explained

```
this.items = [];
```

Stores the queued items.

```
this.counter = 0;
```

Tracks insertion order.

This helps when two items have the same priority.

```
this.items.push({ element, priority, order: this.counter });
```

Adds the item with its priority and arrival order.

```
this.items.sort((a, b) => {
  return a.priority - b.priority || a.order - b.order;
});
```

Sorts by priority first.

If priorities are equal, it sorts by arrival order.

```
return this.items.shift().element;
```

Removes and returns the highest-priority item.

Since the array is sorted, the highest-priority item is at the front.

---

59. Weakness of Sorting Every Time

The sorting version is easy to understand.

But it can be inefficient.

Why?

Every time you add something, the whole array is sorted again.

Sorting can cost:

```
O(n log n)
```

This may be too slow for large priority queues.

That is why heaps are useful.

Simple meaning:

> Sorting every time is simple but not always efficient.

Part J — Heaps

60. What Is a Heap?

A **heap** is a tree-like data structure often used to implement priority queues efficiently.

Simple meaning:

> A heap keeps the highest-priority item easy to access.

There are two common heap types:

Heap type	Rule
Min-heap	Smallest value stays at the top
Max-heap	Largest value stays at the top

---

61. Min-Heap

A min-heap keeps the smallest value at the top.

Example:

```
      1
     /
    3   5
   /
  8   10
```

The smallest value is:

```
1
```

Simple meaning:

> In a min-heap, the parent is smaller than or equal to its children.

Use a min-heap when smaller numbers mean higher priority.

Example:

```
priority 1 = urgent
priority 5 = less urgent
```

---

62. Max-Heap

A max-heap keeps the largest value at the top.

Example:

```
      100
     /
   80     90
  /
 20  50
```

The largest value is:

```
100
```

Simple meaning:

> In a max-heap, the parent is larger than or equal to its children.

Use a max-heap when larger numbers mean higher priority.

---

63. Heap Shape Rule

A heap is usually a **complete binary tree**.

Simple meaning:

> Levels are filled from left to right.

Good heap shape:

```
      1
     /
   3    5
  /
 8   10
```

Weak shape for a heap:

```
      1

        3

          5
```

A heap should not become a long chain.

---

64. Heap Order Rule

For a min-heap:

```
parent <= children
```

For a max-heap:

```
parent >= children
```

Important:

A heap is not fully sorted.

Example min-heap:

```
      1
     /
   3   5
  /
 8   10
```

This does not mean every value is in exact sorted order from left to right.

It only means every parent follows the heap rule with its children.

Simple warning:

> A heap is ordered enough to get the top priority quickly, but it is not a sorted array.

---

65. Heap Stored as an Array

A heap is drawn like a tree, but commonly stored in an array.

Tree:

```
      1
     /
   3   5
  /
 8   10
```

Array:

```
[1, 3, 5, 8, 10]
```

Why does this work?

Because a complete binary tree has predictable positions.

---

66. Heap Index Formulas

For a node at index `i`:

```
parentIndex = Math.floor((i - 1) / 2);
leftChildIndex = 2 * i + 1;
rightChildIndex = 2 * i + 2;
```

Example array:

```
const heap = [1, 3, 5, 8, 10];
```

Indexes:

```
Index:  0  1  2  3   4
Value:  1  3  5  8  10
```

For index 0:

```
left child = 2 * 0 + 1 = 1
right child = 2 * 0 + 2 = 2
```

So value 1 has children:

```
3 and 5
```

For index 1:

```
left child = 2 * 1 + 1 = 3
right child = 2 * 1 + 2 = 4
```

So value 3 has children:

```
8 and 10
```

---

67. Heap Peek

`peek` means look at the top item without removing it.

For a heap stored in an array, the top is index 0.

```
function peek(heap) {
  if (heap.length === 0) {
    return null;
  }

  return heap[0];
}
```

Complexity:

```
O(1)
```

Why?

Because index 0 is directly accessible.

Simple meaning:

> Peek is fast because the highest-priority item is always at the front.

---

68. Heap Push / Insert

When adding a value to a min-heap:

1. Add it to the end of the array.
2. Move it upward until the heap rule is fixed.

This upward movement is often called:

```
bubble up
```

or:

```
heapify up
```

Simple meaning:

> Add at the end, then move up if needed.

Complexity:

```
O(log n)
```

Why?

Because the item moves up through the height of the tree.

---

69. Heap Pop / Remove Top

When removing the top value from a min-heap:

1. Save the root value.
2. Move the last value to the root.
3. Remove the last array item.
4. Move the new root downward until the heap rule is fixed.

This downward movement is often called:

```
bubble down
```

or:

```
heapify down
```

Simple meaning:

> Remove the top, replace it with the last item, then move that item down.

Complexity:

```
O(log n)
```

---

70. Heap Operation Costs

Operation	Meaning	Cost
`peek`	Look at the top item	O(1)
`push`	Add an item and fix the heap	O(log n)
`pop`	Remove the top item and fix the heap	O(log n)
`heapify`	Build or repair a heap	O(n)
Arbitrary search	Find any random value	O(n)
Arbitrary delete	Delete a random value	O(n)
Storage	Store all values	O(n)

Simple meaning:

> Heaps are excellent when you repeatedly need the smallest or largest item. They are not designed for searching any random item quickly.

---

71. Simple Min-Heap Implementation

```
class MinHeap {
  constructor() {
    this.values = [];
  }

  parentIndex(index) {
    return Math.floor((index - 1) / 2);
  }

  leftChildIndex(index) {
    return 2 * index + 1;
  }

  rightChildIndex(index) {
    return 2 * index + 2;
  }

  swap(indexA, indexB) {
    const temp = this.values[indexA];
    this.values[indexA] = this.values[indexB];
    this.values[indexB] = temp;
  }
```

```
peek() {
  if (this.values.length === 0) {
    return null;
  }

  return this.values[0];
}

push(value) {
  this.values.push(value);
  this.bubbleUp();
}

bubbleUp() {
  let index = this.values.length - 1;

  while (index > 0) {
    const parent = this.parentIndex(index);

    if (this.values[parent] <= this.values[index]) {
      break;
    }

    this.swap(parent, index);
    index = parent;
  }
}

pop() {
  if (this.values.length === 0) {
    return null;
  }

  if (this.values.length === 1) {
    return this.values.pop();
  }

  const min = this.values[0];
  this.values[0] = this.values.pop();
  this.bubbleDown();
  return min;
}

bubbleDown() {
  let index = 0;

  while (true) {
```

```
      const left = this.leftChildIndex(index);
      const right = this.rightChildIndex(index);
      let smallest = index;

      if (
        left < this.values.length &&
        this.values[left] < this.values[smallest]
      ) {
        smallest = left;
      }

      if (
        right < this.values.length &&
        this.values[right] < this.values[smallest]
      ) {
        smallest = right;
      }

      if (smallest === index) {
        break;
      }

      this.swap(index, smallest);
      index = smallest;
    }
  }
}

const heap = new MinHeap();
heap.push(5);
heap.push(3);
heap.push(8);
heap.push(1);

console.log(heap.peek());
console.log(heap.pop());
console.log(heap.pop());
console.log(heap.pop());
console.log(heap.pop());
```

Expected output:

```
1
1
3
```

```
5
8
```

---

72. Min-Heap Code Explained Simply

```
this.values = [];
```

The heap is stored as an array.

```
parentIndex(index) {
  return Math.floor((index - 1) / 2);
}
```

Finds the parent of a node.

```
leftChildIndex(index) {
  return 2 * index + 1;
}
```

Finds the left child.

```
rightChildIndex(index) {
  return 2 * index + 2;
}
```

Finds the right child.

```
swap(indexA, indexB) {
```

Swaps two values in the array.

```
push(value) {
  this.values.push(value);
```

```
  this.bubbleUp();
}
```

Adds a value to the end, then moves it upward if needed.

```
bubbleUp()
```

Fixes the heap after insertion.

If the new value is smaller than its parent, it swaps upward.

```
pop()
```

Removes the smallest value from the heap.

```
bubbleDown()
```

Fixes the heap after removing the top value.

If the root is larger than a child, it swaps downward.

Part K — How These Structures Connect

73. Graphs vs Trees

Graph	Tree
General relationship structure	Special hierarchical graph
Can have cycles	Usually no cycles
Can be disconnected	Usually connected from one root
Can have many connection patterns	Parent-child structure

Simple meaning:

> Every tree is graph-like, but not every graph is a tree.

74. Trees vs Binary Search Trees

Tree	Binary Search Tree
General hierarchy	Ordered binary tree
Any number of children, depending on tree type	At most two children
No required value order	Left smaller, right larger

Simple meaning:

> A BST is a special tree designed for searching.

---

75. Tries vs Binary Search Trees

Trie	BST
Stores strings by characters	Stores comparable values
Good for prefixes	Good for ordered search
Search based on word length	Search based on tree height

Simple meaning:

> Use tries for prefix problems. Use BSTs for ordered value search.

---

76. Priority Queues vs Heaps

Priority Queue	Heap
Abstract behavior	Concrete implementation
Says what should happen	Says how to store it efficiently
Remove highest-priority item first	Keeps highest-priority item at the root

Simple meaning:

> A priority queue is the idea. A heap is a common way to build it.

Part L — Common Beginner Mistakes

77. Mistake 1: Traversing a Graph Without `visited`

Bad idea:

```
function badTraversal(graph, node) {
  console.log(node);

  for (const neighbor of graph[node]) {
    badTraversal(graph, neighbor);
  }
}
```

Problem:

> In a cyclic graph, this can repeat forever.

Better:

```
function safeTraversal(graph, node, visited = new Set()) {
  if (visited.has(node)) {
    return;
  }

  visited.add(node);
  console.log(node);

  for (const neighbor of graph[node] ?? []) {
    safeTraversal(graph, neighbor, visited);
  }
}
```

---

78. Mistake 2: Thinking BFS and DFS Always Return the Same Order

BFS and DFS often visit nodes in different orders.

BFS:

```
level by level
```

DFS:

```
deep path first
```

Also, DFS order can change depending on the order of neighbors in the adjacency list.

---

79. Mistake 3: Thinking a Heap Is Fully Sorted

A heap is not a sorted array.

It only guarantees that the top value has priority.

For a min-heap:

```
parent <= children
```

But the whole array is not necessarily sorted.

---

80. Mistake 4: Using a BST Without Keeping It Balanced

A BST can become weak if inserted values are already sorted.

Example insertion order:

```
1, 2, 3, 4, 5
```

Bad shape:

```
1

  2

    3

      4

        5
```

This behaves like a linked list.

81. Mistake 5: Confusing `search()` and `startsWith()` in a Trie

In a trie:

```
search("ca")
```

checks if `"ca"` is a complete word.

```
startsWith("ca")
```

checks if any word starts with `"ca"`.

These are different.

Part M — Practice Tasks

82. Practice Task 1: Represent a Graph

Create this graph:

```
A -- B -- D
|
C
```

Expected JavaScript:

```
const graph = {
  A: ["B", "C"],
  B: ["A", "D"],
  C: ["A"],
  D: ["B"]
};
```

---

83. Practice Task 2: Run BFS

Using this graph:

```
const graph = {
  A: ["B", "C"],
  B: ["A", "D"],
  C: ["A"],
  D: ["B"]
};
```

Run BFS from A.

Expected result:

```
["A", "B", "C", "D"]
```

---

84. Practice Task 3: Run DFS

Using the same graph, run DFS from A.

Possible result:

```
["A", "B", "D", "C"]
```

Note:

> DFS result can vary depending on neighbor order.

---

85. Practice Task 4: Explain BFS vs DFS

Question:

```
Why does BFS use a queue, but DFS can use recursion?
```

Correct idea:

```
BFS waits to process nodes level by level, so it needs a queue.
DFS goes down one path first, so recursion naturally tracks the current path.
```

---

86. Practice Task 5: Debug Broken BFS

What is wrong with this code?

```
function badBfs(graph, start) {
  const queue = [start];

  while (queue.length > 0) {
    const node = queue.shift();
    console.log(node);

    for (const neighbor of graph[node]) {
      queue.push(neighbor);
    }
  }
}
```

Problem:

```
There is no visited Set.
```

In a cyclic graph, this can repeat forever.

Better version:

```
function goodBfs(graph, start) {
  const visited = new Set();
  const queue = [start];

  visited.add(start);

  while (queue.length > 0) {
    const node = queue.shift();
    console.log(node);

    for (const neighbor of graph[node] ?? []) {
      if (!visited.has(neighbor)) {
        visited.add(neighbor);
        queue.push(neighbor);
      }
    }
  }
}
```

87. Practice Task 6: Search a BST

Given this BST:

```
        10
       /
      5    15
     / \
    2   7     20
```

Question:

```
How would you search for 20?
```

Answer:

```
Start at 10.
20 is greater than 10, go right to 15.
20 is greater than 15, go right to 20.
Found it.
```

---

88. Practice Task 7: Trie Prefix Check

Words:

```
cat
car
care
```

Questions:

```
search("ca") -> ?
startsWith("ca") -> ?
search("car") -> ?
```

Answers:

```
search("ca") -> false
startsWith("ca") -> true
search("car") -> true
```

Why?

`"ca"` is a prefix, not a complete word.

---

89. Practice Task 8: Heap Index Formula

Given this heap array:

```
const heap = [1, 3, 5, 8, 10];
```

Question:

```
What are the left and right children of index 1?
```

Formulas:

```
leftChildIndex = 2 * i + 1;
rightChildIndex = 2 * i + 2;
```

For index 1:

```
left = 2 * 1 + 1 = 3
right = 2 * 1 + 2 = 4
```

So the child values are:

```
heap[3] = 8
heap[4] = 10
```

Part N — Sprint 24 Master Checklist

By the end of Sprint 24, you should be able to explain these in simple words.

Concept	You should know
Graph	Nodes connected by edges
Node / Vertex	One item in a graph
Edge	A connection between nodes
Neighbor	A directly connected node
Directed graph	Edges have direction
Undirected graph	Edges go both ways
Weighted graph	Edges have values or costs
Cyclic graph	Graph has a loop
Acyclic graph	Graph has no loop
DAG	Directed graph with no cycles
Disconnected graph	Graph has separate parts
Adjacency list	Each node stores a list of neighbors
Adjacency matrix	Grid of connections
BFS	Queue-based level-by-level traversal
DFS	Stack or recursion-based deep traversal
`visited` Set	Prevents repeated graph visits
Tree	Hierarchical graph-like structure
Root	Top node of a tree
Parent	Node above another node
Child	Node below another node
Leaf	Node with no children
Subtree	Smaller tree inside a tree
Binary tree	Tree where each node has at most two children
BST	Binary tree where left is smaller and right is larger
Trie	Character tree for words and prefixes
Priority queue	Removes by priority, not only arrival order
Min-heap	Smallest value stays at the top
Max-heap	Largest value stays at the top

Concept	You should know
Heap array formulas	Parent and child indexes are calculated from array positions

---

Part O — Final Sprint 24 Summary

Sprint 24 is about **relationships, hierarchy, and priority**.

The core ideas are:

```
Graphs store relationships.
BFS visits level by level using a queue.
DFS goes deep using recursion or a stack.
Trees store hierarchy.
Binary search trees organize values for faster search.
Tries store words by character paths.
Priority queues remove important items first.
Heaps make priority queues efficient.
```

The most important beginner rules are:

1. Use an adjacency list for most beginner graph problems.
2. Use a `visited` Set when traversing graphs.
3. Use BFS when you need level order or shortest path in an unweighted graph.
4. Use DFS when you need deep exploration or recursive tree traversal.
5. Remember that a tree is a hierarchy.
6. Remember that a BST must follow the left-smaller, right-larger rule.
7. Use tries for prefix problems.
8. Use priority queues when importance matters.
9. Use heaps when priority queue operations need to be efficient.
10. Do not confuse a heap with a fully sorted array.

Final simple mental model:

> Use graphs for connections.
> Use trees for hierarchy.
> Use tries for word prefixes.
> Use priority queues for importance.
> Use heaps to make priority queues fast.